CFD Final Project

Surya Ramachandran

1. We have talked about solving Navier-Stokes equations with the Projection Method in class and using 1D case as an example. Now, please expand the formulation to 2D and program a 2D code for the following tasks.

   (a) Please derive the formulation for non-dimensionalized N-S equations and the Projection Method for 2D.
   Ans: We have Navier-Stoked equations for incompressible flow given by:

$$\frac{\partial v}{\partial t} + v.\nabla v = -\frac{1}{\rho}\nabla p + \nu \nabla^2 v$$

   And continuity:

$$\nabla v = 0$$

   Non-dimensionalizing, we get:

$$\frac{\partial v^*}{\partial t^*} + v^*.\nabla v^* = -\frac{1}{\rho^*}\nabla p^* + \nu^* \nabla^2 v^*$$

   Where: $v^* = \frac{v}{V}; t^* = \frac{t}{t_{char}} = \frac{tU}{L}; p^* = \frac{p}{\rho_o U^2}; V, U, L$ are all characteristic quantities.

   For 2-D, we get:

$$\frac{\partial u^*}{\partial t^*} + u^* \frac{\partial u^*}{\partial x^*} + v^* \frac{\partial u^*}{\partial y^*} = \frac{1}{Re}\left(\frac{\partial^2 u^*}{\partial x^{*2}} + \frac{\partial^2 u^*}{\partial y^{*2}}\right)$$

$$\frac{\partial v^*}{\partial t^*} + u^* \frac{\partial v^*}{\partial x^*} + v^* \frac{\partial v^*}{\partial y^*} = \frac{1}{Re}\left(\frac{\partial^2 v^*}{\partial x^{*2}} + \frac{\partial^2 v^*}{\partial y^{*2}}\right)$$

   For intermediate steps of velocity, we have:

$$\frac{u^* - u^n}{\Delta t} = -(u.\nabla)u^n + \frac{1}{Re}\nabla^2 u^n$$

$$u^* = u^n + \Delta t\left[-\left(u^n \frac{\partial u^n}{\partial x} + v^n \frac{\partial u^n}{\partial y}\right) + \frac{1}{Re}\nabla^2 u^n\right]$$

$$v^* = v^n + \Delta t\left[-\left(u^n \frac{\partial v^n}{\partial x} + v^n \frac{\partial v^n}{\partial y}\right) + \frac{1}{Re}\nabla^2 v^n\right]$$

For pressure at each time step:

$$\nabla . u^{n+1} = 0$$

$$\frac{u^{n+1} - u^*}{\Delta t} = -\nabla p^{n+1}$$

$$\nabla \left(\frac{u^{n+1} - u^*}{\Delta t} \right) = -\nabla^2 p^{n+1}$$

And then we correct for velocity at the next full timestep:

$$u^{n+1} = u^* - \Delta t \frac{\partial p^{n+1}}{\partial x}$$

$$v^{n+1} = v^* - \Delta t \frac{\partial p^{n+1}}{\partial y}$$

(b) Please derive the solution for 2D plane Poiseulle flows.
Ans: Poiseulle flow is a incompressible, steady, fully developed flow driven by x-direction pressure gradient. Meaning:
Steady: time independent- $\frac{\partial}{\partial t} = 0$
Fully developed: no spatial changes along x direction- $\frac{\partial u}{\partial x} = 0, v = 0$
Therefore, continuity is verified like so:

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0$$

X-momentum for Navier-Stokes becomes:

$$0 = -\frac{\partial p}{\partial x} + \frac{1}{Re} \frac{\partial^2 u}{\partial y^2}$$

$$Re.G = \frac{\partial^2 u}{\partial y^2}$$

Where $G = \frac{\partial p}{\partial x}$
Integrating the above equation:

$$\frac{\partial u}{\partial x} = Re.G.y.C_1$$

Integrating a second time:

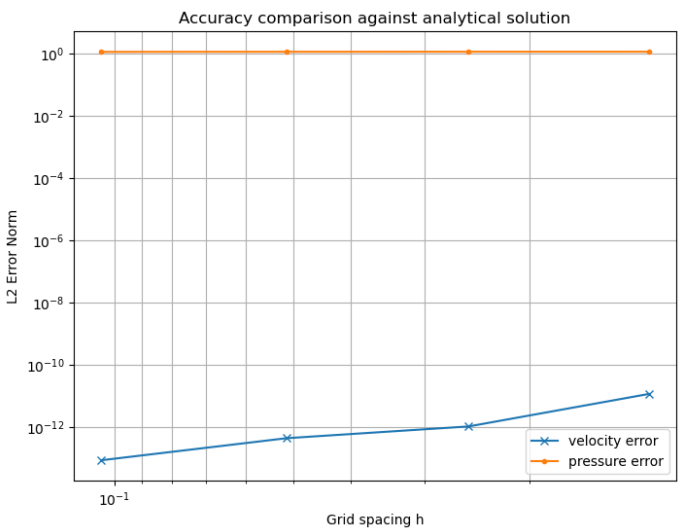
$$u = Re.G.\frac{y^2}{2}.C_1 y + C_2$$

Boundary conditions are given by $y = [-1 : 1]$
Therefore: $u(-1) = 0; u(1) = 0$

Applying boundary conditions to the equation, we get

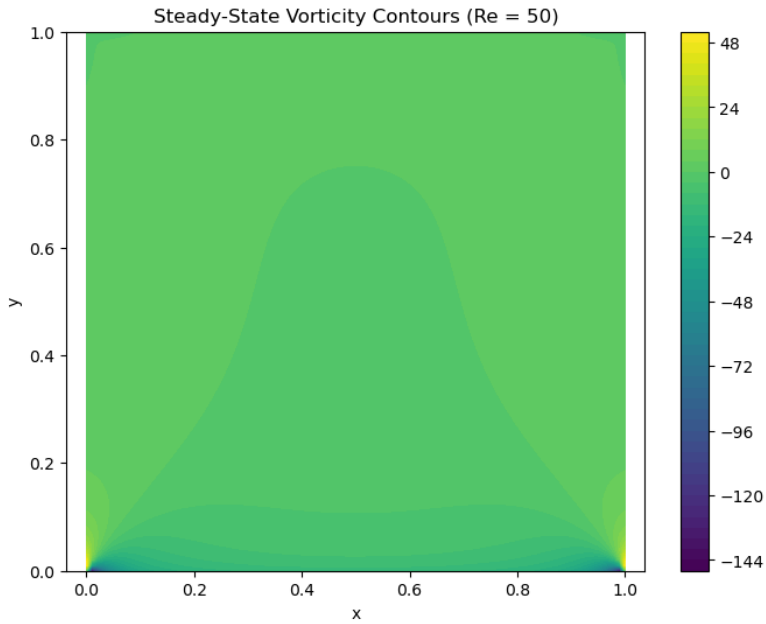
$$u(y) = \frac{Re}{2}\frac{\partial p}{\partial x}(y^2 - 1)$$

(c) Please perform the order accuracy for spacial discretization for both velocity and pressure in your 2D code with the analytical solutions derived in the previous task.
Ans:



For different grid resolutions:
N= 20: velocity error = 8.450e-14, pressure error = 1.140e+00
N= 40: velocity error = 4.330e-13, pressure error = 1.147e+00
N= 80: velocity error = 1.034e-12, pressure error = 1.151e+00
N= 160: velocity error = 1.145e-11, pressure error = 1.153e+00

(d) Please run a case of lid-driven cavity flows with the $Re = 50$ and present the steady state vorticity contour.
Ans:



Iteration 0, max vorticity error = 1.98e+02
Iteration 500, max vorticity error = 3.95e-03
Iteration 1000, max vorticity error = 1.01e-03
Iteration 1500, max vorticity error = 2.41e-04
Iteration 2000, max vorticity error = 5.54e-05
Iteration 2500, max vorticity error = 1.16e-05
Iteration 3000, max vorticity error = 2.39e-06
Converged after 3276 iterations.

Listing 1: Order Accuracy

```
import numpy as np
import matplotlib.pyplot as plt

#defining the analytical solutions
def analytical_u(y, Re=100, G=1):
    return 0.5*Re*G*(y**2-1) #profile for plane poiseuille flow

def analytical_p(x, G=1):
    return -G*x  #pressure decrease along the x-direction

#compute L2 error
def l2_error(numerical, exact):
    return np.sqrt(np.mean((numerical-exact)**2))
```

```python
# Grid convergence study: compare u and p directly to analytical
def convergence_test():
    Re= 100 #reynolds number
    G= 1 #pressure gradient driving the flow
    errors_u= [] #initializing velocity errors
    errors_p= [] #initializing pressure errors
    hs= [] #initializing grid spacings
    resolutions= [20, 40, 80, 160]  #grid resolutions

    for N in resolutions:
        #domain: x in [0, 2], y in [-1, 1]
        Lx= 2 #domain length in x
        Ly= 2 #domain length in y
        dx= Lx/(N-1) #grid spacing in x
        dy= Ly/(N-1) #grid spacing in y
        hs.append(dx) #uniform grid spacing

        x= np.linspace(0,Lx,N) #x-coordinates
        y= np.linspace(-1,1,N) #y-coordinates
        X, Y= np.meshgrid(x,y) #2D grid for X and Y

        u_exact = analytical_u(Y, Re, G) #velocity field
        p_exact = analytical_p(X, G) #pressure field

        #numerical solution using derived 2D solver
        nu = 1/Re
        u_num= np.zeros_like(Y)
        p_num= np.zeros_like(X)

        for i in range(N): #velocity solver
            A= np.zeros((N, N))
            b= np.ones(N)*G/nu
            A[0, 0]= A[-1, -1]= 1
            b[0]= b[-1]= 0
            for j in range(1, N - 1):
                A[j,j-1]= 1/dy**2
                A[j,j]= -2/dy**2
                A[j,j+1]= 1/dy**2
            u_num[:,i]= np.linalg.solve(A,b)

        for j in range(N): #pressure solver
            p= np.zeros(N)
            p[0]= 0  #BC: p(0)=0
        for i in range(1, N):
            p[i]= p[i-1]-G*dx #forward integration
```

```
            p_num[j,:]= p


        error_u = l2_error(u_num, u_exact)
        error_p = l2_error(p_num, p_exact)
        errors_u.append(error_u)  #storing velocity error
        errors_p.append(error_p)  #storing pressure error

        print(f"N= {N}: u error = {error_u:.3e}, p error = {error_p:.3e}")

    plt.figure(figsize=(8,6))
    plt.loglog(hs,errors_u,'x-', label='velocity error')
    plt.loglog(hs,errors_p,'.-', label='pressure error')
    plt.xlabel("Grid spacing h")
    plt.ylabel("L2 Error Norm")
    plt.title("Accuracy comparison against analytical solution")
    plt.grid(True, which='both')
    plt.legend()
    plt.gca().invert_xaxis()
    plt.show()

convergence_test()
```

Listing 2: Lid Driven Cavity

```
    import numpy as np
import matplotlib.pyplot as plt

#simulation parameters
N= 100 #no of grid points in each direction
Re= 50 #reynolds number
L= 1 #domain length
nu= 1/Re #kinematic viscosity
dx=dy= L/(N-1) #grid spacing in each direction
dt= 0.001 #time step size
max_iter= 10000 #max no of time steps for iteration
tol= 1e-6 #tolerance for vorticity change

#grid parameters
x= np.linspace(0,L,N) #x-coordinates
y= np.linspace(0,L,N) #y-coordinates
X,Y= np.meshgrid(x,y) #2D meshgrid for plotting

psi= np.zeros((N, N)) #initialize streamfunction
omega= np.zeros((N, N)) #initialize vorticity

U= 1 #lid velocity (top wall)
```

```
def apply_bc(omega, psi): #spply boundary conditions to vorticity
    omega[0,1:-1]= -2*(psi[0,1:-1]-psi[1,1:-1])/dy**2-2*U/dy #lid
    omega[-1, 1:-1]= -2*(psi[-1, 1:-1]-psi[-2,1:-1])/dy**2 #bottom wall
    omega[1:-1,0]= -2*(psi[1:-1,0]-psi[1:-1,1])/dx**2 #left wall
    omega[1:-1, -1]= -2*(psi[1:-1,-1]-psi[1:-1,-2])/dx**2 #right wall
    return omega

def solve_poisson(omega, psi, iterations=500): #poisson equation
    for _ in range(iterations):
        psi[1:-1,1:-1]= 0.25*(psi[1:-1,2:]+psi[1:-1,:-2]+psi[2:, 1:-1]+
                              psi[:-2, 1:-1]+dx**2*(-omega[1:-1, 1:-1]))
    return psi

def compute_velocity(psi):
    # Compute velocity components from streamfunction
    u = np.zeros_like(psi)                    # x-component of velocity (horizonta
    v = np.zeros_like(psi)                    # y-component of velocity (vertical)
    u[1:-1, 1:-1] = (psi[1:-1, 2:] - psi[1:-1, :-2]) / (2 * dy)
# u =       / y
    v[1:-1, 1:-1] = -(psi[2:, 1:-1] - psi[:-2, 1:-1]) / (2 * dx)
# v = -     / x
    return u, v                               # Return velocity fields

#time integration
for it in range(max_iter): #iterate in time up to max_iter
    psi= solve_poisson(omega,psi,iterations=100) #solve poisson equation
    u,v= compute_velocity(psi) #compute velocity

    #advection + diffusion
    omega_new = omega.copy() #store updated values
    omega_new[1:-1, 1:-1] = (omega[1:-1,1:-1]+dt*(-u[1:-1, 1:-1]*(omega[1:-1,2:]
-omega[1:-1,:-2])/(2*dx)-v[1:-1,1:-1]*(omega[2:,1:-1]-omega[:-2,1:-1])/(2*dy)+nu
((omega[1:-1,2:]-2*omega[1:-1,1:-1]+omega[1:-1,:-2])/dx**2+(omega[2:,1:-1]-2*
omega[1:-1,1:-1]+omega[:-2,1:-1])/dy**2)))

    omega_new= apply_bc(omega_new,psi) #apply BC to new vorticity

    error= np.max(np.abs(omega_new-omega)) #check for convergence
    if it%500== 0: #progress every 500 steps
        print(f"Iteration {it}, max vorticity error = {error:.2e}")
    if error<tol: #stop if solution has converged
        print(f"Converged after {it} iterations.")
        break
```

```
    omega= omega_new #update vorticity field for next step

plt.figure(figsize=(8, 6))
cp = plt.contourf(X, Y, omega, 50, cmap='viridis')
plt.colorbar(cp)
plt.title(f'Steady-State Vorticity Contours (Re = {Re})')
plt.xlabel('x')
plt.ylabel('y')
plt.axis('equal')
plt.show()
```